

Språk och Datorer (729G49)

Språkteknologi 1: Textextraktion och -segmentering

30/3/2026



Innehåll

- Textextraktion
- Textsegmentering och tokenisering
- Regex (reguljära uttryck)
- Statistisk korpusanalys

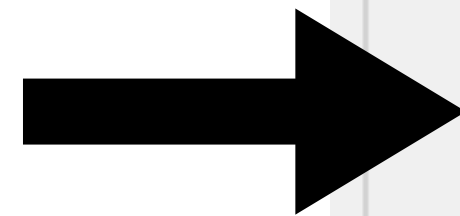
Att bygga en mini-corpus

- För veckans labb vill vi bygga en egen liten korpus till en statistisk analys!
- I denna föreläsning ska vi därför titta på hur man kan:
 - Ladda ner webbsidor.
 - Tokenisera dem med reguljära uttryck.
- Den sista delen handlar om statistiska analyser och lag som ni ska testa i labben.

Extrahera text från webbsidor

“Spindla” webben

- Exempel: Vi vill ha alla Wikipedia-artiklar i en viss kategori.
- https://sv.wikipedia.org/wiki/Kategori:Marsvinsartade_gnagare
- Vi kan skriva en funktion som hittar alla länkar inom kategorin.
- Sedan kan vi ladda ner dem.



```
import requests
from bs4 import BeautifulSoup

category_url = "https://sv.wikipedia.org/wiki/Kategori:Marsvinsar

# Get all article links from a category page
def fetch_category_links(url):
    article_links = []
    while url:
        response = requests.get(url)
        soup = BeautifulSoup(response.text, 'html.parser')

        for link in soup.find_all('a', href=True):
            href = link['href']
            if href.startswith('/wiki/') and not href.startswith(
                article_links.append('https://sv.wikipedia.org' +

        next_page = soup.find('a', {'title': 'Nästa sida'})
        url = 'https://sv.wikipedia.org' + next_page['href'] if n

    return article_links

links = fetch_category_links(category_url)
```

Ladda ner en Wikipedia-artikel

- Okej, vi ska göra det lite lättare och bara ladda ner en artikel:

- <https://sv.wikipedia.org/wiki/Kapybara>

- Enkelt!

```
import requests

url = "https://sv.wikipedia.org/wiki/Kapybara"

response = requests.get(url)

with open("Kapybara_article.html", 'w', encoding='utf-8') as file:
    file.write(response.text)
```


Avformatering

- Hur ser den nedladdade artikeln ut?
 - HTML-uppmärkningar.
- Vi vill bara ha råtext i korpusen.
 - Extrahera sidans egentliga innehåll med hjälp av en HTML-parser.

```
<!DOCTYPE html>
<html class="client-nojs vector-feature-language-in-header-enabled vector-feature-language-in-main-page-
<head>
<meta charset="UTF-8">
<title>Kapybara - Wikipedia</title>
<script>(function(){var className="client-js vector-feature-language-in-header-enabled vector-feature-
"wgDefaultDateFormat":"dmy","wgMonthNames":["","januari","februari","mars","april","maj","juni","juli",
"Wikipediaartiklar med identifierare från Fossilworks","Wikipediaartiklar med identifierare från MSW",
"wgULSisCompactLinksEnabled":false,"wgVector2022LanguageInHeader":true,"wgULSisLanguageSelectorEmpty":
"skins.vector.search.codex.styles":"ready","skins.vector.styles":"ready","skins.vector.icons":"ready",
"ext.popups","ext.visualEditor.desktopArticleTarget.init","ext.visualEditor.targetLoader","ext.echo.cer
<script>(RLQ=window.RLQ||[]).push(function(){mw.loader.impl(function(){return["user.options@12s5i",fun
}]);});</script>
<link rel="stylesheet" href="/w/load.php?lang=sv&modules=ext.cite.styles%7Cext.uls.interlanguage%7
<script async="" src="/w/load.php?lang=sv&modules=startup&only=scripts&raw=1&skin=vector">
<meta name="ResourceLoaderDynamicStyles" content="">
<link rel="stylesheet" href="/w/load.php?lang=sv&modules=ext.gadget.babel%2Cerror%2Cgeo%2Cpintonly
<link rel="stylesheet" href="/w/load.php?lang=sv&modules=site.styles&only=styles&skin=vector">
<meta name="generator" content="MediaWiki 1.44.0-wmf.3">
<meta name="referrer" content="origin">
<meta name="referrer" content="origin-when-cross-origin">
<meta name="robots" content="max-image-preview:standard">
<meta name="format-detection" content="telephone=no">
<meta property="og:image" content="https://upload.wikimedia.org/wikipedia/commons/thumb/6/6e/Kapybara_1
<meta property="og:image:width" content="1200">
<meta property="og:image:height" content="817">
<meta property="og:image" content="https://upload.wikimedia.org/wikipedia/commons/thumb/6/6e/Kapybara_1
<meta property="og:image:width" content="800">
<meta property="og:image:height" content="545">
<meta property="og:image" content="https://upload.wikimedia.org/wikipedia/commons/thumb/6/6e/Kapybara_1
<meta property="og:image:width" content="640">
<meta property="og:image:height" content="436">
<meta name="viewport" content="width=1120">
<meta property="og:title" content="Kapybara - Wikipedia">
<meta property="og:type" content="website">
<link rel="preconnect" href="//upload.wikimedia.org">
<link rel="alternate" media="only screen and (max-width: 640px)" href="//sv.m.wikipedia.org/wiki/Kapyba
<link rel="alternate" type="application/x-wiki" title="Redigera" href="/w/index.php?title=Kapybara&
<link rel="apple-touch-icon" href="/static/apple-touch/wikipedia.png">
<link rel="icon" href="/static/favicon/wikipedia.ico">
<link rel="search" type="application/opensearchdescription+xml" href="/w/rest.php/v1/search" title="Wil
<link rel="EditURI" type="application/rsd+xml" href="//sv.wikipedia.org/w/api.php?action=rsd">
<link rel="canonical" href="https://sv.wikipedia.org/wiki/Kapybara">
<link rel="license" href="https://creativecommons.org/licenses/by-sa/4.0/deed.sv">
```

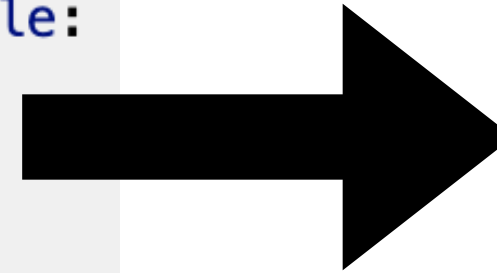

Extrahera alla p-element (paragrafer)

```
from bs4 import BeautifulSoup

with open("Kapybara_article.html", 'r', encoding='utf-8') as file:
    soup = BeautifulSoup(file, 'html.parser')

paragraphs = soup.find_all('p')
cleaned_text = '\n'.join([para.get_text() for para in paragraphs])

with open("Kapybara_article.txt", 'w', encoding='utf-8') as file:
    file.write(cleaned_text.strip())
```



Kapybara eller vattensvin (*Hydrochoerus hydrochaeris*) är världens största gnagare. Den påminner i sitt levnadssätt om flodhästar men är närmare släkt med marsvin. Fram till 1990-talet kategoriserades kapybaran som ensam art i släktet *Hydrochoerus*, som i sin tur placerades som ensamt släkte i familjen *Hydrochaeridae*. I nyare taxonomiska avhandlingar kategoriseras den norra populationen som den egna arten *Hydrochoerus isthmius* och de placeras istället i familjen marsvin (*Caviidae*).[1][2]

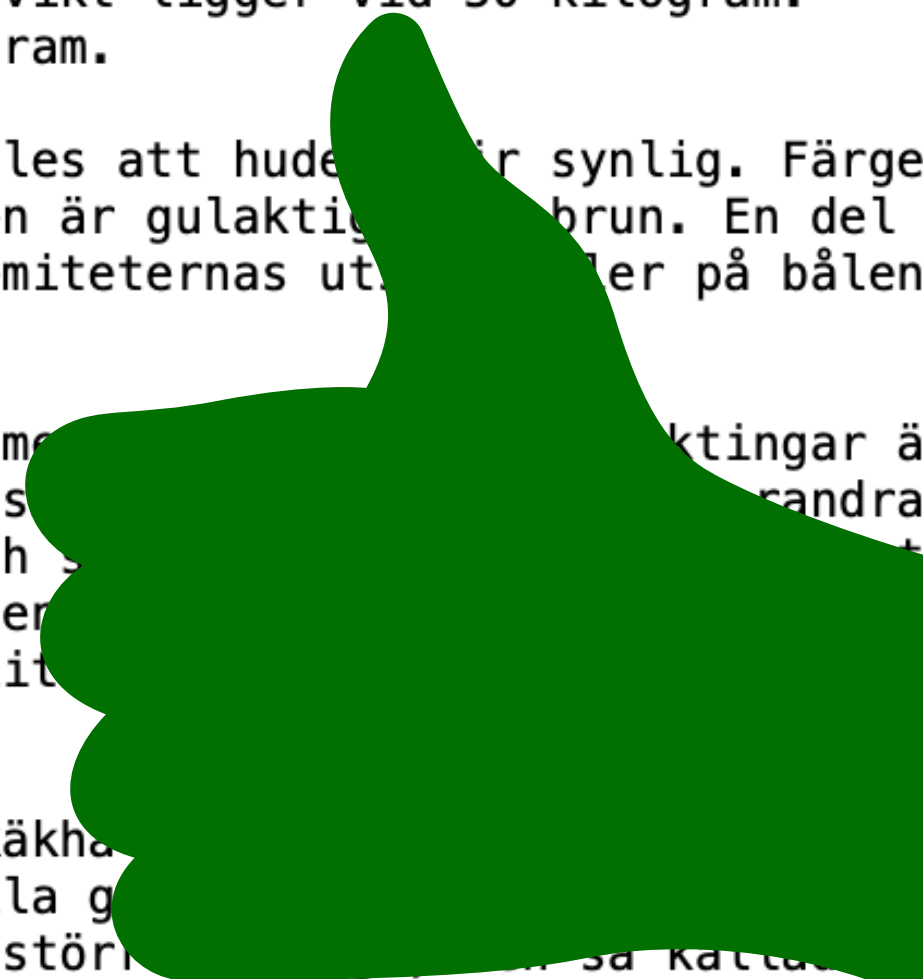
Kapybaran når en kroppslängd upp till 130 centimeter och en vikt upp till 61 kilogram. Arten förekommer i stora delar av Sydamerikas slättland samt i angränsande regioner av Centralamerika. Kapybaran jagas för köttets och hudens skull men räknas inte som en hotad art. Dess närmaste släkting, utöver *Hydrochoerus isthmius*, är klippmarsvinet och listas därför tillsammans som en underfamilj inom familjen marsvin (*Caviidae*).

Djuret har en stor och robust kropp med korta extremiteter. Kapybaran blir upp till 130 cm lång och 60 cm hög (vid skuldran)[3]. Hanar och honor är lika i utseendet, nosen är dock längre på hanar. Hanen har en oval, naken doftkörtel på nosen, vilken avskräddar ett vitt, klibbigt sekret. Båda könen har även analkörtlar vars doft gör att de känner igen varandra. På de främre fötterna finns fyra tår och på de bakre fötterna tre. Tårna liknar hovar och mellan dem finns lite simhud. Svansen är bara rudimentär. Honor är lite större än hanar och de förstnämnda har en genomsnittlig vikt på 61 kilogram medan hanarnas genomsnittliga vikt ligger vid 50 kilogram. Allmänt kan vikten ligga på mellan 27 och 80 kilogram.

Pälsen är lång och grov men vid vissa ställen så gles att huden blir synlig. Färgen varierar mellan rödbrun och grå på ovansidan, buken är gulaktig eller brun. En del individer har svarta fläckar i ansiktet, vid extremiteternas utlöparer på bålarna. Hårens längd ligger mellan 30 och 120 millimeter.

Kapybaran har ett påfallande brett huvud. Jämförd med andra gnagare är öronen för stora. Nosen är förstörd och mera avrundad. Näsborrarna är stora och rundade. Öronen är små och avrundade. Även ögonen är små och runda. Som hos flera andra vattendjur ligger öronen, ögonen och näsan nära varandra i ansiktet[3]. På så sätt behöver de bara lyfta en liten vattenytan för att iakttä omgivningen.

Tandformeln är 1-0-1-3, kapybaran har alltså per käkhalv ett par incisorer och tre molarer, tillsammans 20 tänder. Som hos alla gnagare växer de nya tanderna och växer hela livet. Efter framtänderna finns en stor diastema.



Teckenkodning: En vanlig felkälla

- När vi laddar ner webbsidor måste vi hantera **teckenkodning** korrekt.
 - **UTF-8:** Vanligast på webben, stöder alla språk.
 - **Latin-1:** Äldre kodning som saknar många tecken.
- Fel kodning ger **mojibake**: “ä” → “Ãä”
- Därför: `encoding='utf-8'` i `open()`.

Exempel: Hantera teckenkodning

- **Problem:**

- `response = requests.get(url)`
- `print(response.text[:50])` # "Kapybara År..."

- **Lösning:**

- `response.encoding = 'utf-8'`
- `print(response.text[:50])` # "Kapybara är..." ✓

Etik vid webbskrapning

- **robots.txt:** Talar om vad man får och inte får skrapa.
 - Exempel: <https://sv.wikipedia.org/robots.txt>
- **Rate limiting:** Ladda inte ner för snabbt — använd `time.sleep()` om ni laddar ner många sidor (inte i labben).
- **Upphovsrätt:** Wikipedia är CC BY-SA, andra sidor kan ha restriktioner.
- **GDPR:** Tänk på personuppgifter i texterna ni samlar in.

Segmentering och Tokenisering

Textsegmentering

- **Textsegmentering** är uppgiften att dela upp en text i lingvistiskt meningsfulla enheter, t.ex.:
 - Ord,
 - Morfemer,
 - Meningar,
 - Stycken.

Tokenisering (1)

- När enheterna som segmenteras är ord eller ordliknande enheter så kallas det för **tokenisering**.
- Enklaste fall: Tokenisera på vittecken.

```
>>> text = 'Tokenisera mig!'
>>> text.split()
['Tokenisera', 'mig!']
```
- Men kanske vill vi även separera skiljetecken från ord: ['Tokenisera', 'mig', '!']?
- ...och vill samtidigt behålla 'Karl-Ove' som én token?

Tokenisering (2)

- Och sen finns det språk som inte använder sig av vittecken...
- ...och där tokeniseringen inte alltid är trivial.

A potential overlapping ambiguity in Chinese.

- a. * 布什 在谈话 中指 出
Bush at talk middle-finger out
- b. 布什 在谈话 中 指出
Bush at talk middle point-out
'Bush pointed out in his talk'

Glass et al. 2024:
Language and Computers

Tokenisering: Inte Trivial!

Exempel från
spaCy-biblioteket:

The algorithm can be summarized as follows:

1. Iterate over space-separated substrings.
2. Check whether we have an explicitly defined special case for this substring. If we do, use it.
3. Look for a token match. If there is a match, stop processing and keep this token.
4. Check whether we have an explicitly defined special case for this substring. If we do, use it.
5. Otherwise, try to consume one prefix. If we consumed a prefix, go back to #3, so that the token match and special cases always get priority.
6. If we didn't consume a prefix, try to consume a suffix and then go back to #3.
7. If we can't consume a prefix or a suffix, look for a URL match.
8. If there's no URL match, then look for a special case.
9. Look for "infixes" – stuff like hyphens etc. and split the substring into tokens on all infixes.
10. Once we can't consume any more of the string, handle it as a single token.
11. Make a final pass over the text to check for special cases that include spaces or that were missed due to the incremental processing of affixes.

<https://spacy.io/usage/linguistic-features#tokenization>

Möjliga fel vid tokenisering

- **Undersegmentering**

- Den automatiska tokeniseringen missar att segmentera en teckensekvens som enligt guldstandard ska segmenteras.
- ['Tokenisera', 'mig!'] — ['Tokenisera', 'mig', '!']

- **Översegmentering**

- Den automatiska tokeniseringen delar på en teckensekvens som enligt guldstandard inte ska segmenteras.
- ['S', ':', 't', 'Lars', 'Kyrka'] — ['S:t', 'Lars', 'Kyrka']

Utvärdering av tokeniseringen (1)

- **Guldstandard:** En korpus av högt kvalité, gärna en mänsklig annotation.
 - **Träningsmängden** används för att träna upp tokeniseraren, eller, i vårt fall, utveckla ett reguljärt uttryck.
 - **Testmängden** används för att utvärdera det färdiga systemets förmåga att generalisera utanför träningsmängden.
 - Testmängden får absolut inte vara en del av träningsmängden!

Utvärdering av tokeniseringen (2)

	Hittade vi	Hittade vi inte
Finns i guldstandarden	Sanna Positiva	Falska Negativa
Finns inte i guldstandarden	Falska Positiva	Sanna Negativa

Utvärdering av tokeniseringen (3)

- **Precision:** Hur många procent av alla tokens som vi hittade finns i guldstandarden?

sanna positiva

- $$\frac{\text{sanna positiva}}{\text{sanna positiva} + \text{falska positiva}}$$

- Vår tokeniserare: ['S', ':', 't', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping.']
- Guldstandard: ['S:t', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping', '.']

Utvärdering av tokeniseringen (3)

- **Precision:** Hur många procent av alla tokens som vi hittade finns i guldstandarden?

sanna positiva

- $$\frac{\text{sanna positiva}}{\text{sanna positiva} + \text{falska positiva}}$$

- Vår tokeniserare: ['S', ':', 't', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping.']
- Guldstandard: ['S:t', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping', '.']

$$\frac{4}{4 + 4} = 0.5$$

Utvärdering av tokeniseringen (4)

- **Täckning:** Hur många procent av alla tokens i guldstandardén hittade vi?

sanna positiva

- $\frac{\text{sanna positiva}}{\text{sanna positiva} + \text{falska negativa}}$

- Vår tokeniserare: ['S', ':', 't', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping.']
- Guldstandard: ['S:t', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping', '.']

Utvärdering av tokeniseringen (4)

- **Täckning:** Hur många procent av alla tokens i guldstandardet hittade vi?

sanna positiva

- $\frac{\text{sanna positiva}}{\text{sanna positiva} + \text{falska negativa}}$

- Vår tokeniserare: ['S', ':', 't', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping.']
- Guldstandard: ['S:t', 'Lars', 'Kyrkan', 'är', 'i', 'Linköping', '.']

$$\frac{4}{4 + 3} \approx 0.57$$

Övning: Beräkna precision och täckning

- **Vår tokeniserare:**

- `['Dr.', 'Nilsson', 'bor', 'på', 'Kungsgatan', '12B', '.']`

- **Guldstandard:**

- `['Dr.', 'Nilsson', 'bor', 'på', 'Kungsgatan', '12', 'B', '.']`

- **Frågor:** Finns det exempel på över- eller undersegmentering här? Vilka är sanna positiva? Vad är precision? Täckning?

Exkurs: Subord-tokenisering (BPE)

- Moderna språkmodeller (GPT, BERT) använder inte ordbaserad tokenisering!
- **Byte Pair Encoding (BPE):** Lär sig subord-enheter från data.
 - Slår ihop vanligaste teckenparen iterativt.
 - “tokenisering” → [“token”, “iser”, “ing”]
- **Fördelar:** Hanterar okända ord, begränsar vokabuläret, språkoberoende.

Exempel: BPE steg för steg

- Träningsdata: “hund”, “hundar”, “hundra”
- **Steg 0:** h u n d | h u n d a r | h u n d r a
- **Steg 1:** Vanligaste paret: (h, u) $\rightarrow$ hu
 - hu n d | hu n d a r | hu n d r a
- **Steg 2:** (hu, n) $\rightarrow$ hun
- **Steg 3:** (hun, d) $\rightarrow$ hund
 - hund | hund a r | hund r a

Meningssegmentering

- Ibland vill vi inte bara identifiera ord utan även större enheter så som meningar eller stycken.
 - Meningssegmentering behövs t.ex. för syntaktisk analys
- Meningssegmentering är uppgiften att dela upp en text i meningar.
- Meningssegmentering är svårare än att dela upp en text efter en punkt eller något annat skiljetecken.
 - *I visited the U.S. last year. After that, I went to Canada.*

Exempel: Svåra meningsgränser

- Hur många meningar finns här? Var går gränserna?
 1. “Prof. Andersson fick 3.5 på tentan.”
 2. “Besök www.liu.se. Där finns mer info.”
 3. “Vi åkte till bl.a. Norrköping och Linköping.”

Meningssegmentering: Lösningar

spaCy provides four alternatives for sentence segmentation:

1. [Dependency parser](#): the statistical `DependencyParser`  provides the most accurate sentence boundaries based on full dependency parses.
2. [Statistical sentence segmenter](#): the statistical `SentenceRecognizer`  is a simpler and faster alternative to the parser that only sets sentence boundaries.
3. [Rule-based pipeline component](#): the rule-based `Sentencizer`  sets sentence boundaries using a customizable list of sentence-final punctuation.
4. [Custom function](#): your own custom function added to the processing pipeline can set sentence boundaries by writing to `Token.is_sent_start`.

<https://spacy.io/usage/linguistic-features#sbd>

Regex (reguljära uttryck)

Regex | Reguljära Uttryck | RE

- Ett verktyg för att matcha och manipulera text baserat på specifika mönster.
- Används ofta för att söka, extrahera, eller ersätta text.

```
import re

text = "Min e-post är test@example.com och mitt postnummer är 12345."

regex_mail = r'[\w_-]+@[\w_-]+\.[A-Za-z]{2,}'
emails = re.findall(regex_mail, text)
print("E-postadresser:", emails)
```

✓ 0.0s

E-postadresser: ['test@example.com']

Varning

- Det är ovanligt att någon förstå hur man använder regex från en förklaring.
 - Ni måste testa det!
 - Det absolut bästa är att gå genom uppgifterna på <https://regexone.com/> och testa vad händer med olika uttryck på <https://regex101.com/>.
- Här ska jag främst samla den viktigaste informationen som behövs för labben.

Kom ihåg: Tokenisering

- När enheterna som segmenteras är ord eller ordliknande enheter så kallas det för **tokenisering**.
- Enklaste fall: Tokenisera på vittecken.

```
>>> text = 'Tokenisera mig!'
>>> text.split()
['Tokenisera', 'mig!']
```
- Men kanske vill vi även separera skiljetecken från ord: ['Tokenisera', 'mig', '!']?
- ...och vill samtidigt behålla "Karl-Ove" som én token?

Tokenisering med reguljära uttryck

- När man måste tokenisera en text på ett visst sätt kan man använda **reguljära uttryck**.
- Mönster som man matchar mot en text och som kan anpassas till sina behov.

```
[>>> pattern = r'\S+'  
[>>> re.findall(pattern, text)  
['Tokenisera', 'mig!']
```

```
[>>> pattern = r'\w+'  
[>>> re.findall(pattern, text)  
['Tokenisera', 'mig']
```

```
[>>> pattern = r'\w+|\S'  
[>>> re.findall(pattern, text)  
['Tokenisera', 'mig', '!']
```


Syntax

Hur skapar jag en regex?

Ett tecken matchar sig själv.	a	S:t Lars Kyrka
...samma gäller en sekvens av tecken.	banan	Jag äter en banan för att jag älskar bananer.
Ett lodstreck () är ett logiskt ELLER	a å ä	Måns äter bananer.
Hakparenteser []: Matchar ett av tecknen innanför hakarna	S[oö]der	Söder Soder Sder Soöder
Vanliga parenteser skapar en grupp.	S(o oe ö)der	Söder Soeder Sder Soöder
Cirkumflex (^) tar inversen av en teckenklass	S[^oö]der	Söder Suder Spder

Operatörer

- Följande operatörer skrivs direkt efter det som ska upprepas.
 - **?** Matchar 0 eller 1: `colou?r —> colour | color`
 - ***** Matchar 0 eller flera: `Go*gle —> Ggle | Gogle | Google | Goooogle | ...`
 - **+** Matchar minst 1: `Go+gle —> Gogle | Google | Goooogle | ...`
 - **{2,4}** Matchar 2-4: `Go{2,4}gle —> Google | Goooogle | Goooooogle`

Speciella teckensekvenser

- `\w` Alla alfanumeriska symboler: `[A-Za-z0-9_]`
- `\W` Allt förutom dem alfanumeriska symbolerna: `[^A-Za-z0-9_]`
- `\s` Det vita tecknet
- `\S` Allt förutom det vita tecknet
- `\d` Alla siffror: `[0-9]`
- `\D` Allt förutom siffrorna: `[^0-9]`

Matcha operatortecken

- För att matcha någon av de speciella tecken (`{ } [ ] ( ) ^ $ . | * + ? \`) behöver man en backslash:
 - Punkten `.` skulle matcha alla möjliga tecken. Om man vill bara matcha själva punkten måste man skriva: `\.`

Användning i Python

re

Table of Contents

- re** — Regular expression operations
 - Regular Expression Syntax
 - Module Contents
 - Flags
 - Functions
 - Exceptions
 - Regular Expression Objects
 - Match Objects
 - Regular Expression Examples
 - Checking for a Pair
 - Simulating scanf()
 - search() vs. match()
 - Making a Phonebook
 - Text Munging
 - Finding all Adverbs
 - Finding all Adverbs and their Positions
 - Raw String Notation
 - Writing a Tokenizer

Previous topic

string — Common string operations

re — Regular expression operations

Source code: [Lib/re/](#)

This module provides regular expression matching operations similar to those found in Perl.

Both patterns and strings to be searched can be Unicode strings ([str](#)) as well as 8-bit strings ([bytes](#)). However, Unicode strings and 8-bit strings cannot be mixed: that is, you cannot match a Unicode string with a bytes pattern or vice-versa; similarly, when asking for a substitution, the replacement string must be of the same type as both the pattern and the search string.

Regular expressions use the backslash character ('\\') to indicate special forms or to allow special characters to be used without invoking their special meaning. This collides with Python's usage of the same character for the same purpose in string literals; for example, to match a literal backslash, one might have to write '\\\\\\\\' as the pattern string, because the regular expression must be '\\\\', and each backslash must be expressed as '\\\\' inside a regular Python string literal. Also, please note that any invalid escape sequences in Python's usage of the backslash in string literals now generate a [SyntaxWarning](#) and in the future this will become a [SyntaxError](#). This behaviour will happen even if it is a valid escape sequence for a regular expression.

- Modulen re finns i Pythons standardbibliotek
- Dokumentation: <https://docs.python.org/3/library/re.html>

Funktioner i re

- `re.match(pattern, string)`: Matchar *början* av strängen mot ett mönster. Om matchning hittas returneras ett matchobjekt, annars *None*.
- `re.search(pattern, string)`: Söker efter det första stället där mönstret matchar *någonstans* i strängen. Returnerar ett matchobjekt eller *None*.
- **`re.findall(pattern, string)`: Returnerar en lista med *alla* förekomster av mönstret i strängen.**
- `re.sub(pattern, replacement, string)`: Ersätter alla matchningar av mönstret i strängen med *replacement*.

re.findall()

```
import re

text = "Min e-post är test@example.com och mitt postnummer är 12345. \
      Min första epostaddress var blah-blah@anothertest.se"

regex_mail = r'[\w_-]+@[\w_-]+\.[A-Za-z]{2,}'
emails = re.findall(regex_mail, text)
print("E-postadresser:", emails)
```

✓ 0.0s

E-postadresser: ['test@example.com', 'blah-blah@anothertest.se']

Mail-Regex: Saknas det något?

- `regex_mail = r'[\w_-]+@[ \w_-]+\.[A-Za-z]{2,}'`

- `[\w_-]+`

[test@example.com](#)

- `@`

[test@student.liu.se](#)

- `[ \w_-]+`

[user@example.co.uk](#)

- `\.`

[user_name@company.org](#)

- `[A-Za-z]{2,}`

[first.last@gmail.com](#)

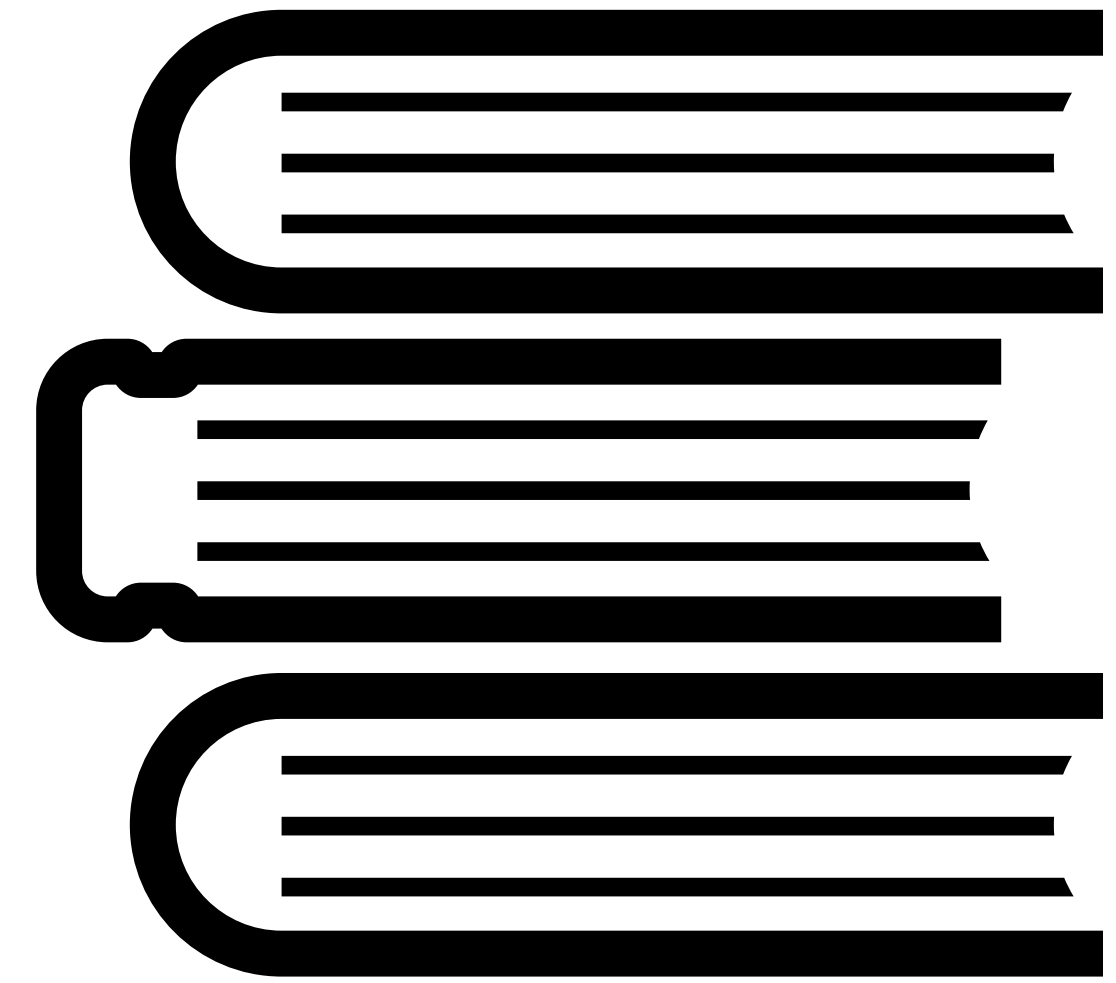
[anna123@liu.se](#)

Statistisk korpusanalys

Att jobba med en korpus

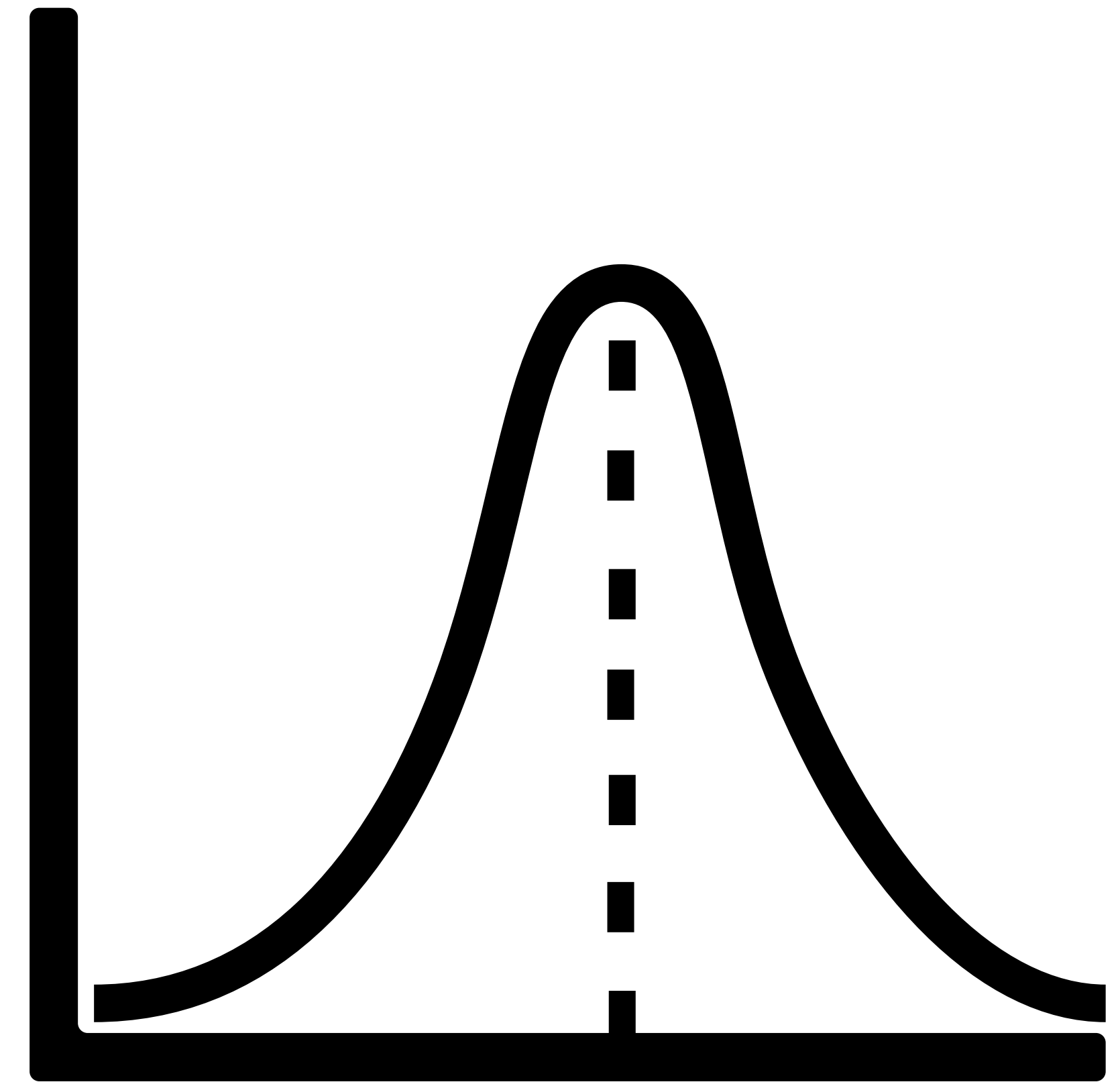
Viktiga frågor att ställa sig (även i labben):

- Hur stor är korpusen?
- ...och hur stor måste den vara för uppgiften?
- Hur divers är korpusen?
- Hur representativ är korpusen, eller vad är den representativ för?



Statistiska Evaluationer

- Enklast: Titta på ord i en korpus.
 - Hur många finns det?
 - ...och hur många olika finns det?
 - Hur är de fördelade?
- ...men vad är det ens, ett ord?



Type Token Ratio (1)

A rose is a rose is a rose

(Gertrude Stein)

- Tre ordformer/types (*a, rose, is*).
- Åtta ord/tokens (*A, rose, is, a, rose, is, a, rose*).
- TTR: $\frac{3}{8} + 0.375$

Type Token Ratio (2)

- Vad betyder en högre TTR?
 - Mer **variation**.
 - Högre **komplexitet**.
- TTR i konversationer brukar vara lägre än i text.
 - Varför?

Normalisering och stoppord

- **Normalisering:** Ska “Kapybara” och “kapybara” räknas som samma type?
 - **Lowercasing:** `text.lower()` — enkelt men förlorar info (“Apple” vs “apple”).
- **Stoppord:** Högfrekventa ord (“och”, “i”, “att”) — jämför Zipf!
 - Ofta borttagna vid informationssökning och textklassificering.
 - (Men för frekvensanalyser vill vi ha kvar dem.)
- **OBS:** Dessa val påverkar TTR och alla statistiska analyser!

Exempel: Normalisering och TTR

- Text: “Kapybaran är stor, kapybaran är snabb.”
- **Utan normalisering:**
 - Types: {Kapybaran, är, stor, kapybaran, snabb} = 5
 - Tokens: 6 $\rightarrow$ TTR = $5/6 \approx 0.83$
- **Med lowercasing:**
 - Types: {kapybaran, är, stor, snabb} = 4
 - Tokens: 6 $\rightarrow$ TTR = $4/6 \approx 0.67$

N-gram och kollokationer

- **N-gram:** Sekvens av n på varandra följande tokens.
 - **Unigram:** “kapybaran”, “är”, “stor”
 - **Bigram:** “kapybaran är”, “är stor”
 - **Trigram:** “kapybaran är stor”
- **Kollokationer:** Ordpar som förekommer oftare än förväntat.
 - “New York”, “i alla fall”, “göra slut”

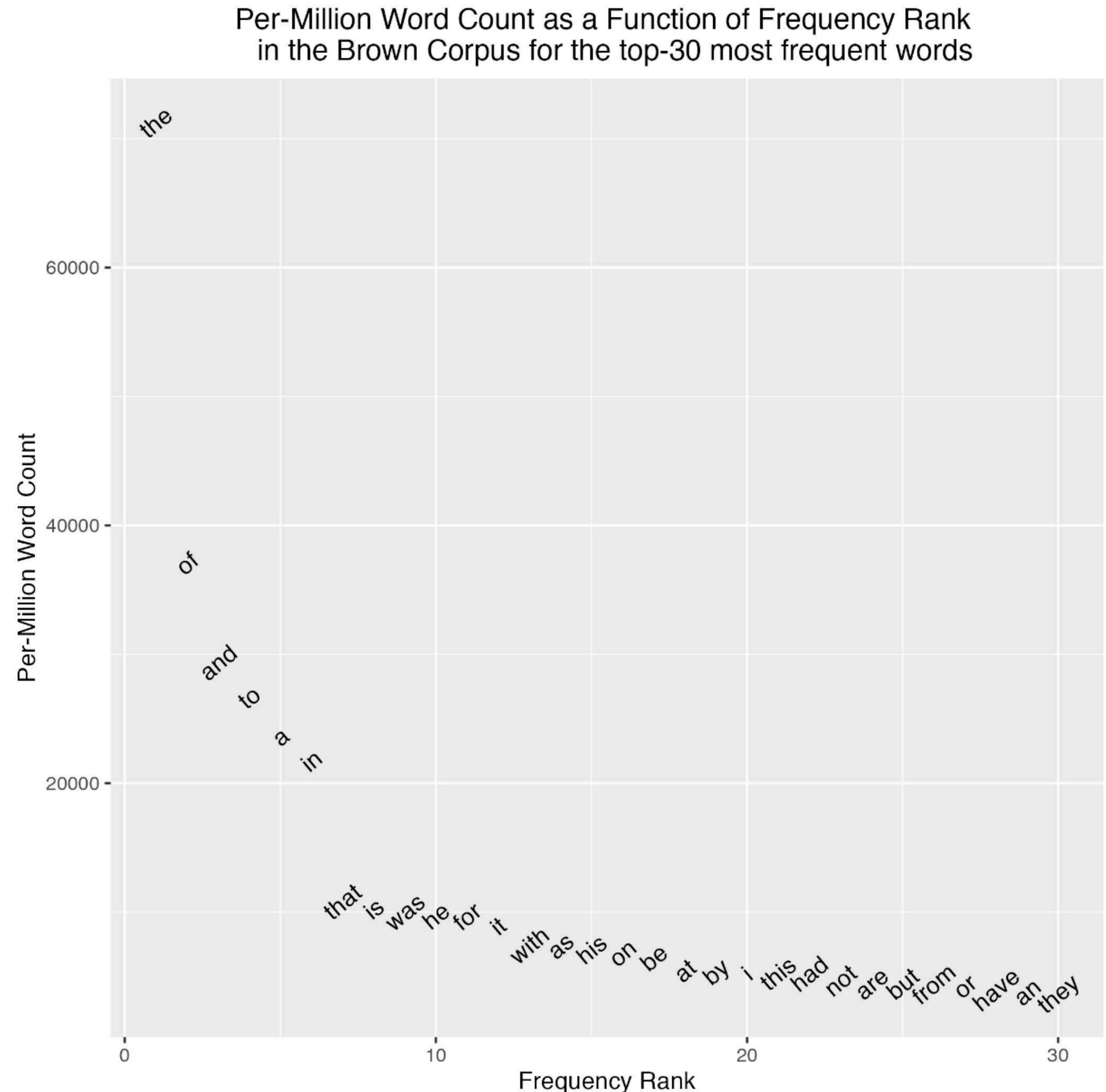
Zipf's Power Law (1)

- Frekvensen av ett visst ord är **omvänt proportionell** mot ordets frekvensranking.
- Det mest frekventa ordet inträffar ungefär dubbelt så ofta som det näst vanligaste ordet, tre gånger så ofta som det tredje vanligaste ordet, etc.

- $$\text{Frekvens} \propto \frac{1}{\text{Frekvensranking}}$$

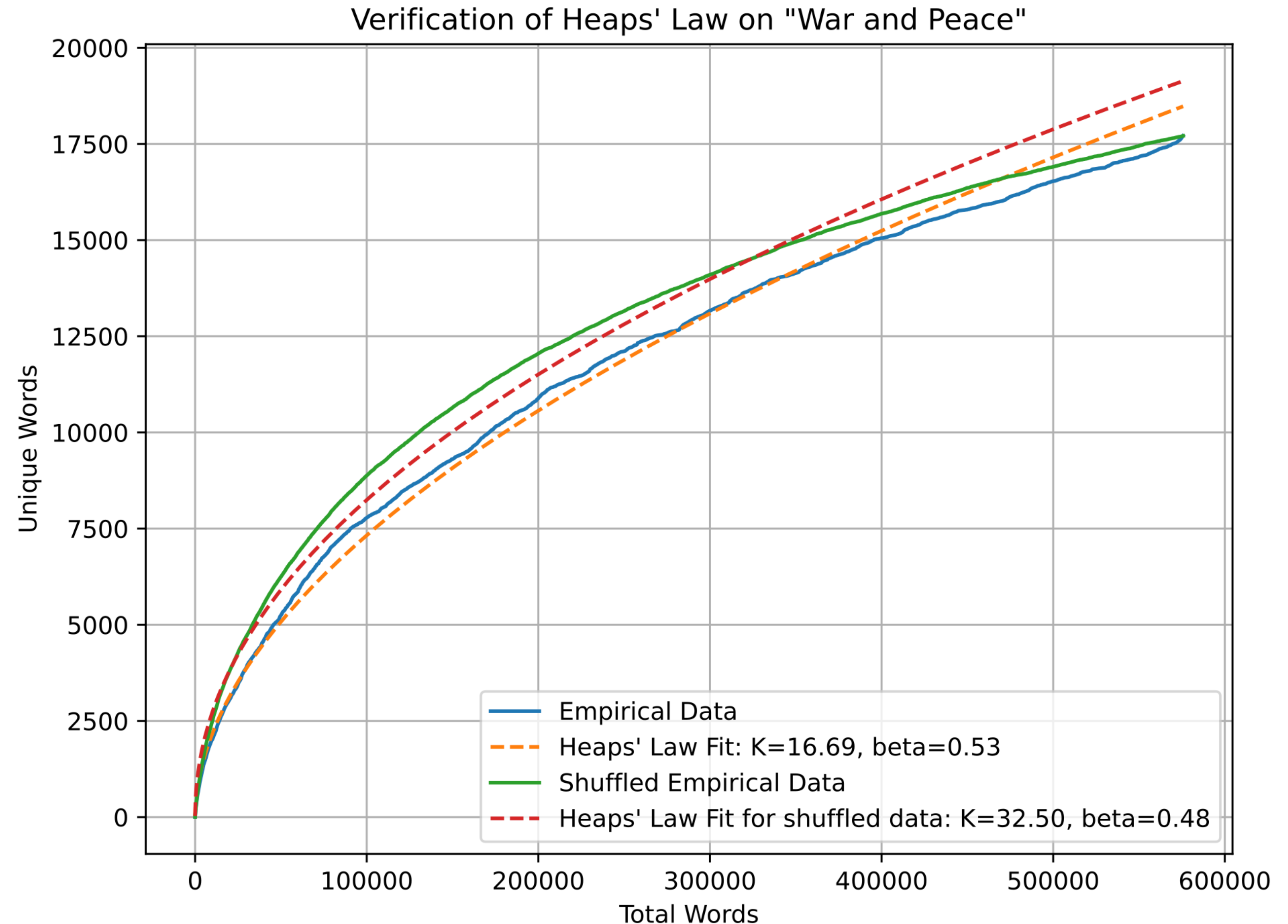
Zipf's Power Law (2)

- I Browns korpusen (modern engelska):
 - "the" är det mest frekventa ordet med en förekomst på 6,9%,
 - "of" står för drygt 3,6%,
 - "and" står för 2,8%.



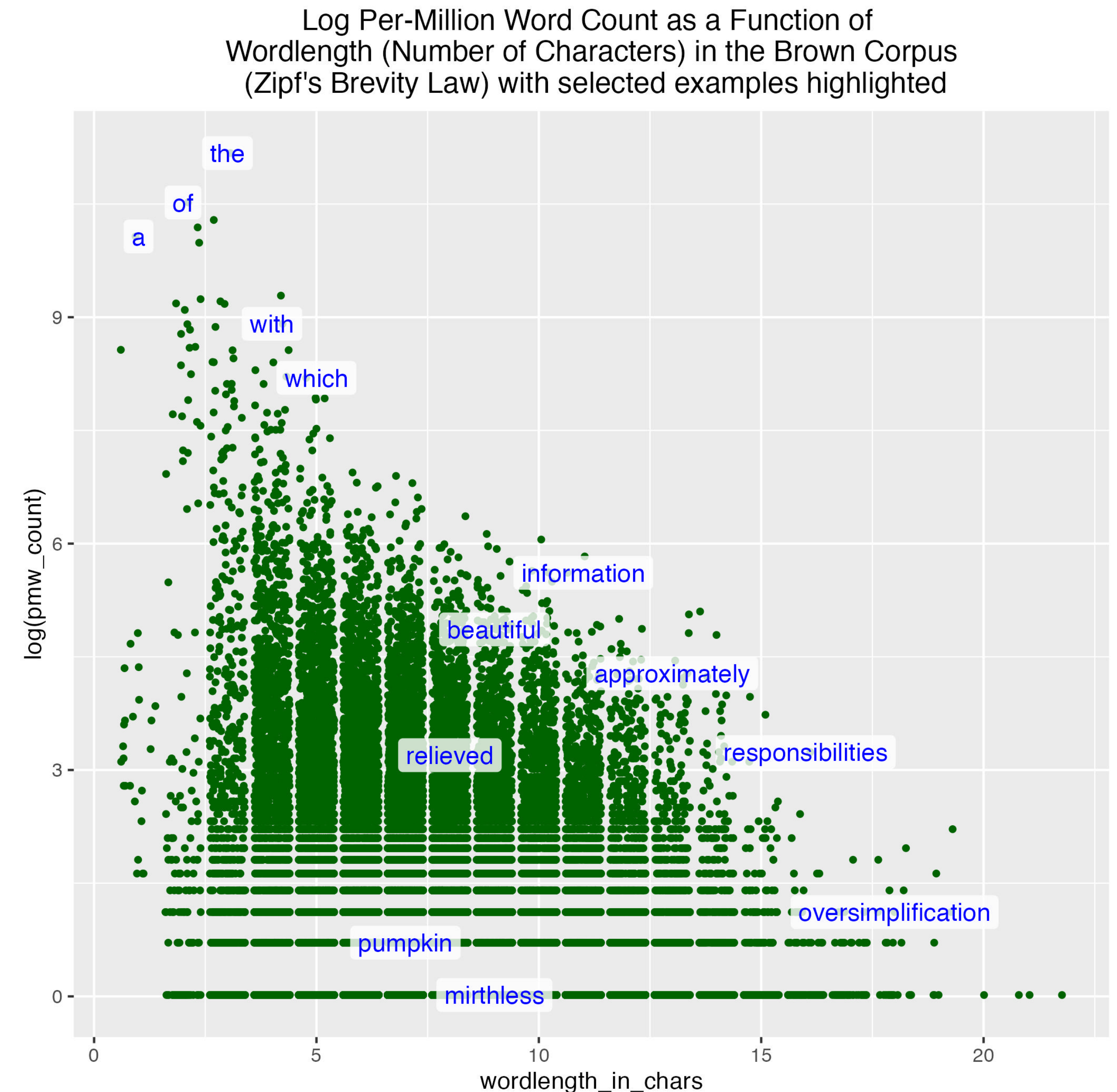
Heap's Law

- Antalet ordformer i en korpus är en funktion av korpusens storlek.
- I Brown korpusen förekommer 38% av alla ordformer bara én gång!
- 14% förekommer två gånger.



Zipf's Brevity Law

- Mer frekventa ord brukar vara kortare.
- Mindre vanliga ord brukar vara längre.
- De vanligaste orden i Brown Corpus: *the*, *be* (i sina olika former), *to*, *of*, *a*.



L1

Tack för idag!

Glöm inte att göra uppgifterna
på <https://regexone.com/>!